Resume Parser and Job Application Chatbot

Assignment 101: AI-Powered Resume Parser and Job Application Chatbot
Submitted to: Conversive.ai

This document describes the implementation of Ava, a conversational assistant that guides a candidate through a job application: role discovery, resume upload, profile extraction, qualification, and contact confirmation.

---

1. Models and libraries

Layer	Technology	Role in the system
Frontend	React 18, Vite, Tailwind CSS	Chat interface, openings catalogue, resume upload
Backend	FastAPI (Python 3.9+)	REST APIs for session, chat, upload, and roles
Persistence	MySQL, SQLAlchemy, PyMySQL	Job roles and candidate records
PDF / text	pdfplumber; UTF-8 file read	Raw resume text before language processing
Primary LLM	Claude via OpenRouter	Chat orchestration (tool calling) and structured resume parse
Fallback LLM	Google Gemini (`google-generativeai`)	Used when OpenRouter is unavailable or out of credits
Tests	pytest, FastAPI TestClient	Scoring, parser helpers, and API checks (LLM mocked)

The assignment permits spaCy, Rasa, or a transformer-based approach. This project uses large language models rather than a separately trained NER or dialogue engine. Claude is invoked through the Anthropic-compatible Messages API (tools + JSON-style extraction). Gemini is a second provider with a chain of API keys.

The language model is limited to two responsibilities:

1. **Dialogue** — select tools (`list_job_roles` , job-description lookup, resume request, contact preference) and generate the user-facing reply.
2. **Extraction** — map resume text to name, email, phone, skills, education, and years of

experience.

Qualification is **not** delegated to the model. It is computed in application code so the decision remains deterministic and auditable.

---

2. Qualification logic

After extraction, the candidate is scored against the role they applied for (Developer, Tester, or Data Analyst). Each role defines required skills and a minimum experience requirement.

Skill overlap = (required skills present on the résumé) / (number of required skills).
Matching is case-insensitive and uses a small alias map (for example `JS` → JavaScript, `MySQL` → SQL). Additional skills beyond the required list do not increase the ratio above 1.0.

Experience score = min(candidate years / role minimum, 1.0).
Years above the minimum do not inflate the score.

Confidence

```
confidence = 0.6 × skill_overlap + 0.4 × experience_score
```

A candidate **qualifies** only when both conditions hold:

- `confidence ≥ 0.5`
- `skill_overlap ≥ 0.5`

The second condition prevents a high year count from compensating for an incomplete skill set. For example, two of five Developer skills and five years of experience yields confidence 0.64, but skill overlap of 0.40 fails the skill floor, so the application is not accepted for that role.

If the chosen role does not qualify, the system proposes only other openings that pass the same rule. If none qualify, the candidate is informed that they were not selected; the assistant does not imply recruiter follow-up. If they qualify, extracted contact details are shown and consent to contact is requested.

---

3. Challenges and how they were addressed

Conversational flexibility. An initial design used keyword and intent rules (a lightweight, non-LLM “NLP” layer). That approach is brittle: users paraphrase, interrupt the happy path, or ask for a job description mid-application. Maintaining a rule tree for those cases was not practical. The production flow uses LLM tool calling for dialogue actions, while matching

remains a closed-form score so "qualified / not qualified" does not depend on model judgement.

Provider and quota limits. Development used free-tier endpoints. OpenRouter returns payment/credit errors when the balance is exhausted; Gemini keys hit quota independently. A single key was insufficient for a stable demonstration. The client therefore tries Claude on OpenRouter first, then Gemini, rotating through multiple Gemini keys when a key is rejected or rate-limited. Chat and resume parse share this path.

Mixed provider history. After a fallback, one session contained Anthropic-shaped and Gemini-shaped messages. Replaying that history caused type errors on the next turn. Message history is now normalised to a common structure before each call so a provider switch does not reset the conversation.

Irregular resume input. Some PDFs yield little usable text; some model responses wrap JSON in markdown fences. The parser retries without JSON mode, strips fences, and rejects empty extracts rather than storing a false profile. Text resumes (`.txt`) are accepted in line with the assignment's PDF-or-text requirement.